

Recherche et Sauvetage Autonomes

Projet B - IA712 Robotique Mobile, Projet Final

Équipe RobotZ

Julien GIMENEZ Hugo FANCHINI Paul CINTRA Yimou ZHANG

<https://github.com/YiZeems/autonomous-search-and-rescue>

IA712 - Séance L18

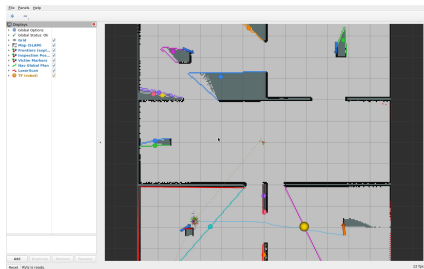
ROS 2 Humble · Ignition Gazebo Fortress · TurtleBot 4 · Nav2 · slam_toolbox · BehaviorTree.CPP

La mission : recherche & sauvetage autonomes

Scénario : un robot explore une zone sinistrée inconnue, trouve des « victimes » (AprilTags), et rapporte leurs coordonnées dans le repère global - *sans intervention humaine*. **Exigences que nous satisfaisons** :

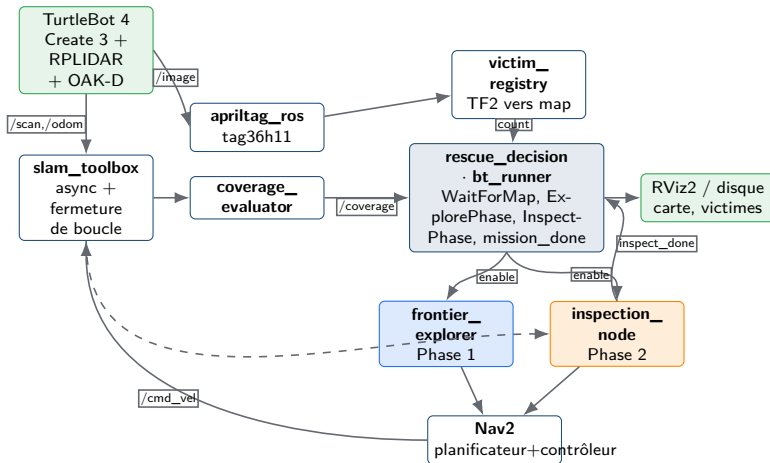
- Exploration par frontières autonome, carte $> 90\%$
- Détection caméra vers map via tf2
- Qualité de carte / fermeture de boucle (SLAM)
- **Décision = Arbre de Comportement, pas FSM**
- Lancement en un clic
- Bonus : glouton vs. gain d'information

Résultat : un run autonome continu - **97,35 % de couverture, 4/4 victimes**.



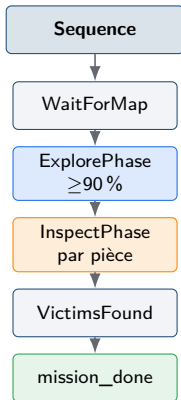
RViz en direct : carte + 4 victimes + algorithmes.

Architecture du système - quatre paquets ROS 2

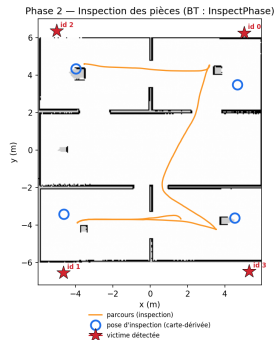
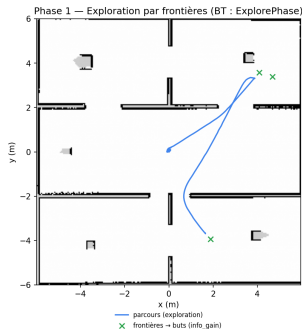


L'Arbre de Comportement orchestre : il cadence l'explorateur (Phase 1) et l'inspecteur (Phase 2) via les topics `/mission/*` ; les nœuds Python font le travail. Le SLAM alimente la carte vers l'exploration, la couverture, Nav2 et l'inspection.

Mission en deux phases orchestrée par l'Arbre de Comportement



Algorithmes en action — mission orchestrée par le Behavior Tree

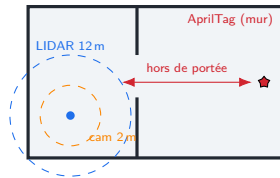


Phase 1 (bleu) l'exploration par frontières construit la carte ; **Phase 2** (orange) inspecte chaque pièce à des poses *dérivées de la carte* (sans coordonnées des victimes) pour capter chaque tag mural.

Pourquoi deux phases ? L'exploration pure cartographie les pièces depuis les portes mais la caméra ne s'approche jamais à moins de 2 m des tags muraux, donc elle plafonne à ~ 2 victimes.

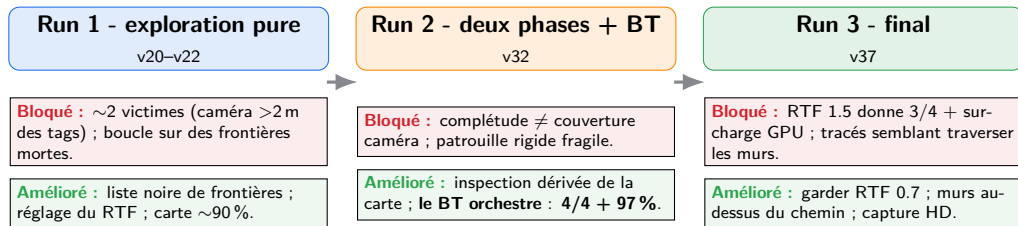
Fondements théoriques

- **SLAM + fermeture de boucle** - carte en ligne cohérente.
- **Exploration par frontières** - limite libre/inconnu ; *gain d'information*
 $g(f) = \text{gain} - \lambda \text{coût}$. Liste noire des frontières mortes (échec Nav2, ou 3× sans gain, écartée).
- **Planification de chemin** - Dijkstra/NavFn vs. A* ; NavFn conservé (tolérant en espace étroit).
- **TF2** - projection détection caméra vers le repère global map (exigence du projet).
- **Arbre de Comportement** - réactif, composable ; Sequence à mémoire, pas une FSM.



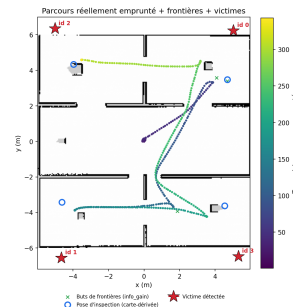
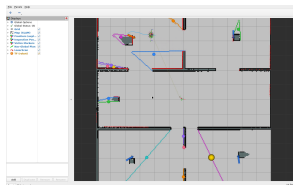
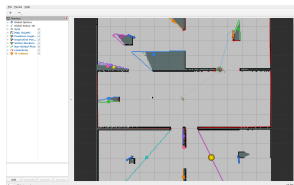
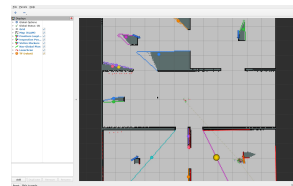
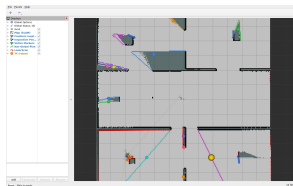
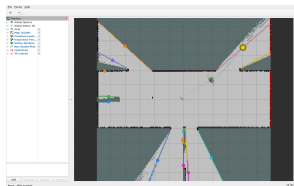
L'écart caméra-vs-LIDAR que la phase d'inspection comble.

Le parcours de décision - trois runs



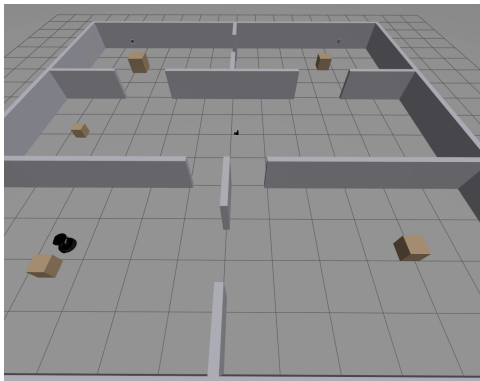
Deux causes racines cachées : le coût Nav2 décourageait l'*entrée* dans les pièces (couverture) ; un RTF effondré *affamait la caméra* (détection). Le BT à 2 phases a ensuite rendu le 4/4 fiable.

Résultats - algorithmes en direct dans RViz



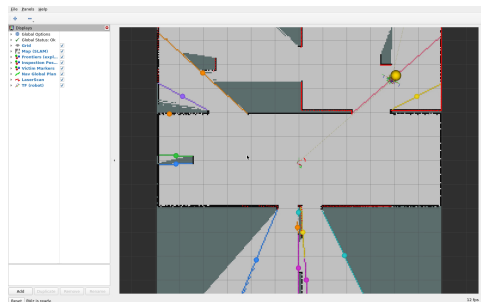
Exploration, inspection, quatre victimes ; en bas à droite : le chemin réel (repère carte, murs tracés par-dessus - il enfile les portes, aucun segment ne traverse un mur).

Voir l'algorithme : Gazebo vs RViz



Gazebo - l'arène simulée : murs, victimes (AprilTags) et le robot.

Chaque *cluster* de frontières a sa couleur, le robot vise la meilleure utilité $g = \text{gain} - \lambda \text{coût}$, et l'on voit le liseré grignoter l'inconnu jusqu'à $> 90\%$. Captures : gazebo_capture.mp4 & rviz_capture.mp4.



RViz - la carte SLAM + les *cellules de frontière colorées par cluster*, la frontière-but (jaune) et le plan Nav2 (bleu).

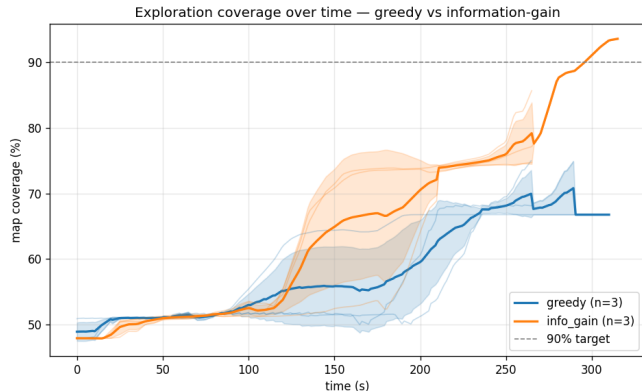
Démo RobotZ



Vidéo de la mission complète (run autonome, un clic) :

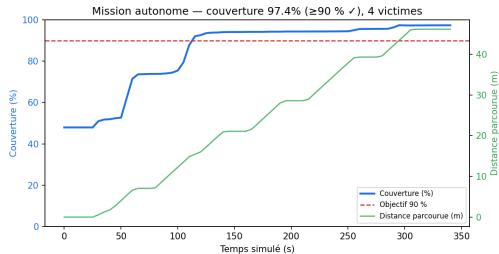
<https://www.youtube.com/watch?v=BKXprWFloFs>

Bonus : couverture cartographique - glouton vs gain d'information



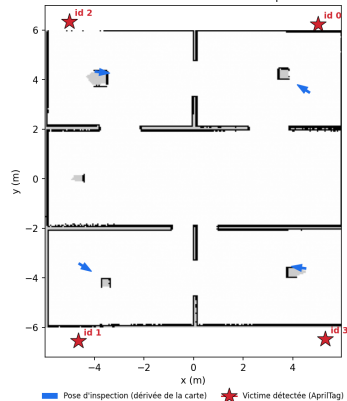
Benchmark $n=3$: le **gain d'information** (orange) atteint la cible 90 % ; le **glouton** (bleu) plafonne ~67–70 % sans jamais l'atteindre - il s'engage à ouvrir des pièces entières (plus d'inconnu par mètre), là où le glouton grignote localement et s'essouffle.

Résultats - couverture, carte et conformité



Couverture & chemin en fonction du temps, cible 90 %.

Carte SLAM finale — 4 victimes + tournée d'inspection autonome



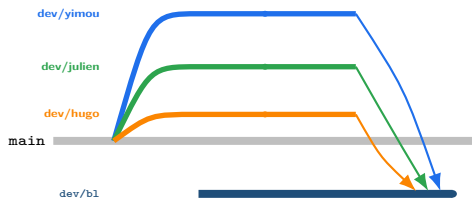
Carte finale : 4 victimes + tournée d'inspection autonome.

97,35 % de couverture · 4/4 victimes · carte annotée · trajectoire enregistrée - un clic, entièrement autonome.

Équipe et collaboration

Branches dev/* par membre fusionnées dans dev/b1 (intégration) :

- **Yimou** - exploration, SLAM/Nav2, cœur de perception
- **Julien** - décision BT, infra (WSL/GPU/RTF), résultats, intégration
- **Hugo** - monde / arène / cibles AprilTag
- **Paul** - support perception/détection, tests



Trois axes de travail vers l'intégration b1.

Leçons apprises

- **Mesurer avant de prescrire** - les causes « évidentes » (mémoire, Windows Update) étaient fausses deux fois ; les journaux d'événements ont trouvé les vraies (TDR GPU, une caméra affamée).
- **Un correctif en révèle un autre** - la détection de victimes a traversé six causes empilées.
- **Robuste l'emporte sur rigide** - exploration adaptative + balayage 360° ; et l'inspecteur récupère une pose qu'une autre machine n'atteint pas (attendre le costmap, se rabattre dans la pièce) - *robustesse inter-machines*.
- **Conformité par conception** - un BT qui *orchestre* (pas seulement supervise) respecte l'énoncé dans l'esprit, pas seulement à la lettre.

Chronologie : L13 lancement · L14 architecture · L15 nœuds/SLAM · L16 BT+perception · L17 intégration/benchmark · L18 deux phases, run final, rapport.

Merci - Questions ?

Nous remercions chaleureusement **Zhi Yan**, enseignant-chercheur à l'ENSTA,
pour le cours IA712 et son accompagnement.

`https://github.com/YiZeems/autonomous-search-and-rescue`

Julien GIMENEZ · Hugo FANCHINI · Paul CINTRA · Yimou ZHANG